

Machine-Checked Translation Adequacy for the Brahmakāṇḍa of Bhartṛhari's *Vākyapadīya*: Commentary-Driven Verification of a Sanskrit Philosophical Text in Lean 4

Sharath Sathish

Independent Researcher

sharath.sathish@gmail.com

<https://github.com/SharathSPHD/vakya-vallari>

July 2026

Abstract

Translations of dense philosophical Sanskrit are under-determined. A single *kārikā* admits several grammatically valid renderings whose philosophical commitments differ radically, and the commentarial tradition, not surface grammar, fixes the intended reading. This paper introduces commentary-driven translation validation (C-TV), an adaptation of translation validation from compiler verification to natural language, and applies it to the complete Brahmakāṇḍa (Book I, 144 verse units) of Bhartṛhari's *Vākyapadīya*. A machine-produced English translation and analytical commentary, prepared with K. A. Subramania Iyer's edition and translation of Book I with the *Vṛtti* as the primary reference text, is compiled verse by verse into a semantic contract: sorted entities, identity, relation, and predication claims, and explicit denials, with every axiom citing the commentary verbatim. A generator compiles each contract into Lean 4, where a decidable adequacy predicate proves that the accepted translation satisfies the contract (144 adequacy theorems) and that 350 plausible mistranslations provably fail (153 contradicted by commentary denials, 197 unlicensed by the axioms), for 549 machine-checked theorems with zero **sorry**. The contracts were authored by an agent swarm gated by a validator and by the Lean kernel; the gates caught errors at every scale, including fabricated citations in quota-degraded agent output and cross-contract ontological drift, which adjudication resolved into a registry of 41 polysemous terms with 91 textually justified senses. An embedding audit of translation-commentary pairs provides a ranked review queue and is reported with its limits. The verification boundary is stated precisely: Lean checks the internal coherence of the formalization against the commentary-derived contract, and the contract-commentary link remains human-auditable through the verbatim citations. All artifacts, including the corpus, contracts, kernel, proofs, and edition, are public and reproducible.

1 Introduction

The *Vākyapadīya* (Treatise on Sentence and Word, c. 5th century CE) is the foundational text of Indian philosophy of language. Its opening verse already exhibits the problem this paper addresses:

*anādinidhanam brahma śabdātattvam yadākṣaram /
vivartate'rthabhāvena prakriyā jagato yataḥ* // (VP 1.1)

A translator must decide whether *śabdatattva* (word-essence) is a property of language or the Absolute itself; whether *vivartate* means that the world is a transformation of Brahman (*pariṇāma*) or its appearance without transformation (*vivarta*); and whether *akṣara* means imperishable, phoneme, or, as the grammarian tradition insists, deliberately both. Each choice yields a grammatical English sentence. The choices differ philosophically as much as materialism differs from idealism. What disambiguates them is not syntax but the commentarial apparatus of *vṛtti* and *ṭīkā* that fixes, verse by verse, which reading the text licenses [Subramania Iyer, 1965, Houben, 2016].

Translation of such texts has therefore been a discipline whose constraints, though real, live in the philologist’s head. This paper makes a subset of those constraints mechanical. The claim is not that a theorem prover can read Sanskrit. The claim is that once interpretive commitments are stated as data, a proof assistant can hold a translation to them exhaustively, incrementally, and publicly.

Scope. The verified object is the *Brahmakāṇḍa*, the first of the text’s three books, in its entirety: 144 verse units, of which 56 carry contested readings. Book I contains the text’s core metaphysics of language (*śabdādvaita* and the *sphoṭa* doctrine) and is the book for which Subramania Iyer [1965] provides the edition with the *Vṛtti* that served as the primary reference for the translation and commentary formalized here. Books II and III exist in the released corpus as unverified context and are outside the scope of the present claims.

Contributions.

1. **Commentary-driven translation validation (C-TV).** The paper adapts translation validation from compiler verification [Pnueli et al., 1998, Leroy, 2009, Tristan and Leroy, 2008] to natural language. The commentary is compiled into per-verse semantic contracts (§3), and a translation’s reading is verified against the contract rather than against the bare source string. To the author’s knowledge this is the first application of an interactive theorem prover to translation adequacy in linguistics, and the first mechanized treatment of a Sanskrit philosophical text.¹
2. **A verified *Brahmakāṇḍa*.** All 144 verse units of Book I carry compiled adequacy theorems, and 350 plausible mistranslations are compiled counterexamples, refuted by theorem rather than by footnote (§6, §9).
3. **An anti-fabrication architecture for agentic formalization.** The contracts were authored by an LLM agent swarm, and every layer is gated: axioms must cite the commentary verbatim; rejected readings must provably fail; entity sorts must be globally consistent or adjudicated as polysemy in a justified registry (§7, §8). The paper reports what the gates caught, including fabricated citations in quota-degraded agent output, as evidence that the architecture rather than agent reliability carries the epistemic load.
4. **A ranked semantic review queue.** An embedding audit of translation–commentary pairs yields a similarity-ranked queue for editorial attention, reported together with a negative finding about its discriminative power at book scale (§10).

¹Prior mechanizations of Indian logic formalize the logical systems (§2); none, to the author’s knowledge, verifies translations of texts.

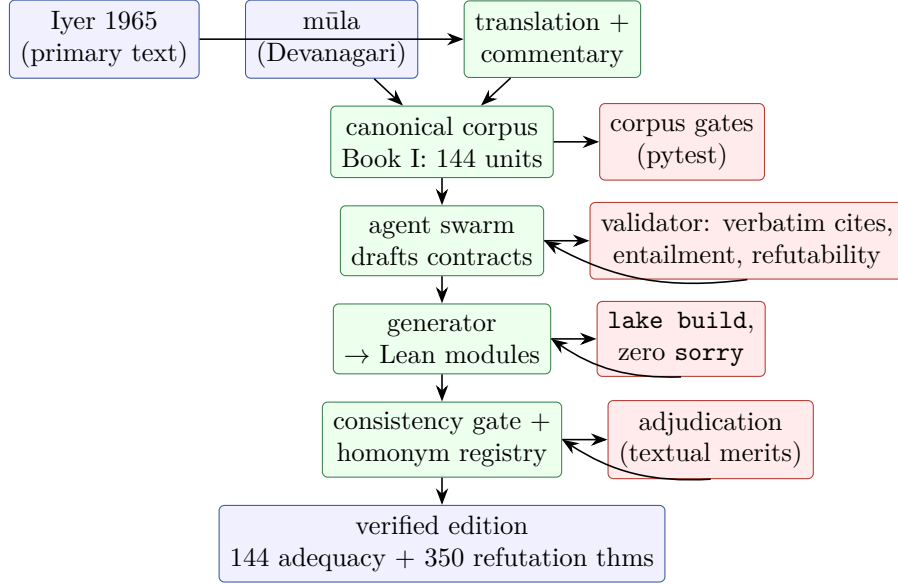


Figure 1: The C-TV pipeline. Every layer has a mechanical gate (red). Failures feed back as repair work, and an append-only research ledger records each draft, rejection, repair, and acceptance.

2 Background and Related Work

2.1 Bhartṛhari and the Brahmakāṇḍa

The *Vākyapadīya* comprises three books (Brahmakāṇḍa, Vākyakāṇḍa, Padakāṇḍa). Book I develops śabdādvaita, the thesis that the Absolute is of the nature of the word, and the sphoṭa doctrine: the meaning-bearing word is an undivided whole, merely manifested by the temporal sequence of sounds [Coward, 1980, Biardeau, 1964, Staal, 1969]. Both doctrines generate exactly the translation hazards the present system checks: sort errors (demoting the Absolute to a property), relation errors (rendering manifestation as transformation), and homonym errors (collapsing the technical senses of akṣara, sphoṭa, and dhvani). The standard modern edition of Book I with the Vṛtti and an English translation is Subramania Iyer [1965]; Houben [1995] treats Bhartṛhari’s theory of relation, which motivates the kernel’s nested relations (§6).

2.2 Computational Sanskrit

Pāṇinian grammar has long been recognized as a formal generative system [Kiparsky, 2009, Cardona, 1976, Scharf, 2009], and a mature computational ecosystem exists for segmentation, morphology, and generation [Goyal et al., 2012, Hellwig and Nehrdich, 2018, Ambuda Project, 2024], with TEI-encoded editions in SARIT [SARIT Project, 2024]. That work is orthogonal and complementary: the present system verifies semantics against commentary, not surface analysis.

2.3 Formalizing Indian logic

Navya-Nyāya has been studied logically since Ingalls [1951] and Matilal [1968]. Recently Panday and Ghosh [2026] encoded its core constructs (relational nesting, delimiters, typed absence) in cubical type theory, Sathish [2026b] trained LLMs on its six-phase epistemic method, and Diehl [2024] mechanized the Buddhist catuṣkoṭi in Lean. These works formalize the logics; none verifies

translations of a text. The kernel here borrows Navya-Nyāya’s structural insights in plain Lean 4 [de Moura and Ullrich, 2021], without cubical machinery.

2.4 Translation validation and autoformalization

Translation validation checks each compiler run a posteriori instead of verifying the compiler [Pnueli et al., 1998, Tristan and Leroy, 2008]. The present system transplants the pattern: the translator is the untrusted optimizer, the contract is the semantic reference, and Lean is the validator. On the authoring side, LLM autoformalization [Wu et al., 2022, Pathak, 2024] supplies the untrusted draft whose output the kernel gates. The loop is an instance of the generate-and-check paradigm with a strict checker.

2.5 Gate-audited agentic research programmes

The formalization loop follows a discipline developed in two companion programmes. Active Circuit Discovery frames mechanistic interpretability under a budget as an experimental-design problem and delegates experiment selection to an active-inference agent [Sathish et al., 2026]. The Prabodha programme runs a pre-registered, gate-audited research loop in which every claim must pass both a code gate and a domain gate, with an append-only evidence ledger and adversarial review [Sathish, 2026a]. The present system inherits the pattern: agents propose, mechanical gates dispose, and an append-only ledger records every draft, rejection, repair, and acceptance. The difference is the checker. Where those programmes gate on statistical criteria, this one gates on a type-checker, so acceptance is a theorem rather than a threshold.

3 Method: Commentary-Driven Translation Validation

3.1 Problem statement

Let v be a verse with source text s_v , commentary c_v , and candidate translation t_v . Direct verification of t_v against s_v is not well posed: the source under-determines the reading, and the commentary is what fixes it. C-TV therefore verifies t_v against a formal object derived from c_v .

Definition 1 (Semantic contract). *A contract for verse v is a JSON document with (i) entities: named terms with ontological sorts drawn from $\{\text{absolute, power, manifestation, linguisticItem, property, cognition}\}$; (ii) axioms and denials: identity, relation, and predication claims over those entities, each carrying a citation that must occur verbatim in c_v ; (iii) an accepted reading: the claims that t_v asserts, each citing t_v verbatim; and (iv) rejected readings: plausible mistranslations rendered as claim sets, each annotated with the failure mode it must exhibit.*

Definition 2 (Adequacy and failure modes). *A reading r is adequate for contract C when every claim of r is entailed by the axioms of C and no claim of r is denied by C . A rejected reading must fail in one of two registered modes: **contradicted** (it asserts a denial) or **unlicensed** (it asserts a claim no axiom entails). Both modes are decidable (§6).*

3.2 The anti-fabrication gate

The verbatim-citation rule is the load-bearing constraint. An axiom that cannot point to the commentary text that licenses it is rejected mechanically, before any proof is attempted. This converts the usual free-form risk of LLM-authored formal content (invented support) into a

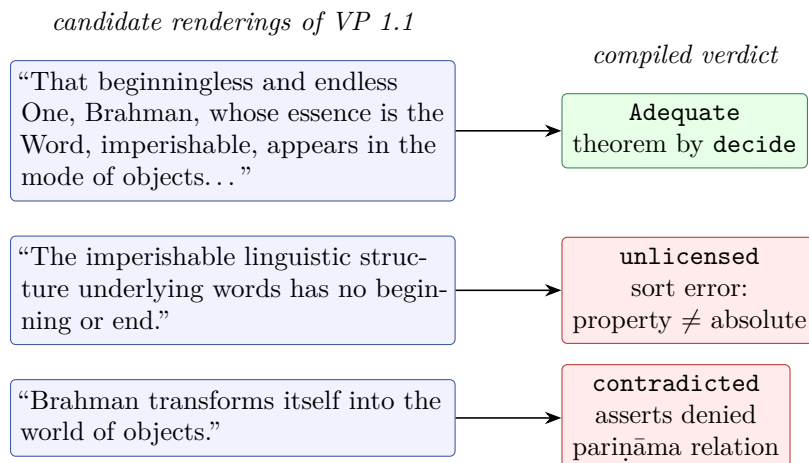


Figure 2: Three renderings of VP 1.1 and their compiled verdicts. The accepted reading is proved adequate; each rejected reading is refuted by a theorem in its registered failure mode.

string-matching check. The same rule applies to the accepted reading, whose claims must cite the translation verbatim, so the contract cannot silently attribute to the translation a claim it does not make.

3.3 Worked example: VP 1.1

Figure 2 shows the compiled fate of three candidate renderings of the opening verse. The contract for 1.1 declares three entities (brahman and śabdatattva of sort absolute, jagat of sort manifestation), four axioms (the identity of śabdatattva and brahman, the vivarta relation to the world, and the predications anādinidhana and akṣara), and one denial (the pariṇāma relation). The accepted rendering asserts four claims, each citing the translation text. Two rejected renderings are registered. The first demotes śabdatattva to a property of language; its identity claim then crosses sorts and no axiom entails it, so it fails as **unlicensed**, with an auxiliary theorem recording the sort error. The second renders vivartate as transformation; it asserts exactly the denied pariṇāma relation and fails as **contradicted**.

3.4 Verification boundary

Lean verifies the internal coherence of the formalization: the accepted reading satisfies the contract and the rivals provably do not. Whether the contract is faithful to Bhartṛhari is not a theorem and cannot be one. That link is kept auditable by construction: every axiom carries the exact commentary sentence it derives from, so checking a contract against the commentary is a lookup rather than a reconstruction. The division of labor is deliberate. The prover keeps commitments; the philologist judges them.

4 The Corpus

4.1 Sources

The edition is built from two sources. The Devanagari mūla kārīkās were scraped from a public edition (wisdomlib.org). The English translation and analytical commentary were produced afresh

Table 1: The corpus and the verified scope. Books II and III are released as unverified context; all verification claims in this paper concern Book I.

	Book I	Book II	Book III	Total
Verse units (canonical)	144	426	1,226	1,796
Contested readings (prose notes)	56	167	63	286
Semantic contracts	144	—	—	144
Adequacy theorems (Lean)	144	—	—	144
Refutation theorems (Lean)	350	—	—	350

for this project by a machine-assisted process working from the Sanskrit, with the edition and annotated translation of Book I with the Vṛtti by [Subramania Iyer \[1965\]](#) as the primary reference text. Interpretive decisions in the commentary follow the Vṛtti as presented there; disputed readings are recorded in a per-verse *contested* note rather than silently resolved.

4.2 Alignment and gates

Alignment between mūla and commentary is checked, never assumed. The commentary units are the canonical keys; every unit must match a mūla entry by exact label or be quarantined with a reason. The build is test-driven, and the tests forced three philologically real discoveries: the *contested* field is a prose note explaining each disputed reading, not a boolean; verse markers legitimately embed Latin half-verse letters; and the source edition assigns one label to two different pages, a defect the commentary itself documents, which the loader merges and flags rather than silently overwriting. The resulting JSONL corpus carries per-record provenance and a manifest with a SHA-256 checksum. Table 1 summarizes the corpus and the verified subset.

5 Contract Anatomy and Statistics

Across the 144 contracts there are 772 entity declarations, 610 axioms, 178 denials, 449 accepted-reading claims, and 350 rejected readings. Most verses carry two or three rejected readings (75 and 58 verses respectively; six carry one and five carry four). Figure 3 shows the distribution of entity sorts and axiom kinds. Relations dominate the axioms (312 of 610), which reflects the text: Book I argues chiefly about how things stand to one another (manifestation, expression, dependence), not about what things are called.

The validator checks, per contract: schema validity; sort membership; ASCII identifier hygiene (a Lean requirement discovered during the loop, §7); verbatim presence of every citation; entailment of the accepted reading; and provable failure of every rejected reading in its registered mode. The last two checks run the same decision procedure the Lean kernel uses, so Python-side acceptance predicts Lean-side theoremhood.

6 The Lean Kernel

6.1 Design

The kernel ([VakyaVallari](#), Lean 4.32, no external dependencies) is deliberately small: the ontology and adequacy core is under 150 lines. Entities are sort-tagged terms; claims are inductive data; a

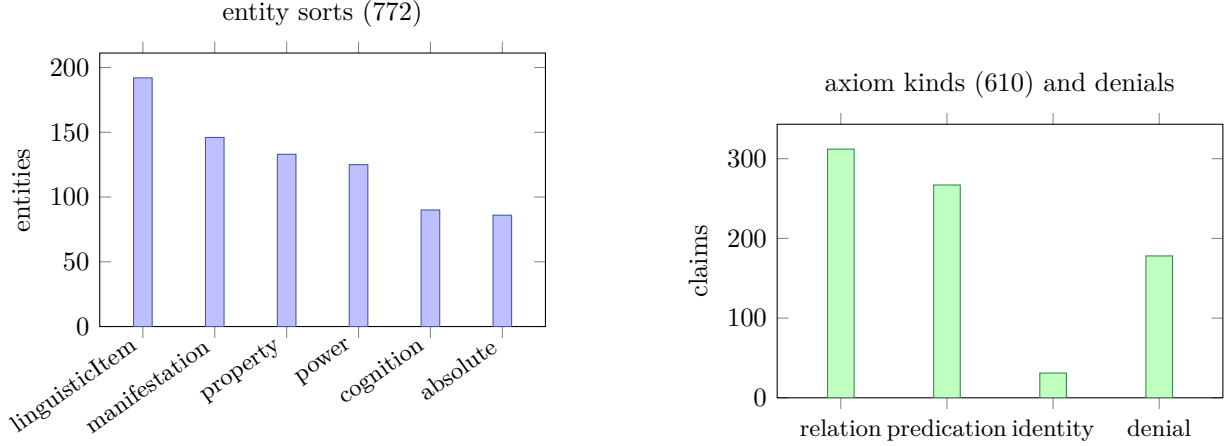


Figure 3: Contract statistics over the 144 Brahmakāṇḍa contracts. Left: entity declarations by ontological sort. Right: axioms by kind, with denials shown alongside.

contract is axioms plus denials; and adequacy is decidable, so every verse-level theorem closes by `decide`. The proof term is the computation.

```

inductive Sorta | absolute | power | manifestation
                  | linguisticItem | property | cognition
structure Entity where name : String; sort : Sorta
-- paramparā: a relation can itself be a relatum, to any depth
inductive Node | ent : Entity → Node
                | rel : String → Node → Node → Node
structure Abhava where pratiyogin : Node; adhikarana : Node
structure Contract where axioms : List Claim; denials : List Claim
def Adequate (c : Contract) (r : Reading) : Prop :=
  r.claims.all (entails c.axioms) ∧ ¬ r.claims.any (denied c.denials)

```

Listing 1: Core of the adequacy kernel (abridged).

6.2 Navya-Nyāya structure

Three design choices are drawn from Navya-Nyāya analysis [Ingalls, 1951, Matilal, 1968]. *Sorted identity*: identity claims are stateable across sorts but decidable false when sorts differ, so a mistranslation that demotes the Absolute to a property fails by sort error, the C-TV failure mode realized as a computation. *Nested relations*: `Node` makes relations first-class relata (paramparā-sambandha, cf. Houben, 1995), so a claim such as “time is the instrumental cause of the manifestation-relation” is representable without ad hoc reification. *Typed absence*: `Abhava` carries its counterpositive and locus as data, and the kernel proves the Navya-Nyāya involution (the absence of an absence computes to the presence) as a definitional fact.

6.3 Generated modules

Each contract compiles to a Lean module in which the accepted reading’s adequacy is a theorem and every rejected reading’s failure is a theorem. Sort-error refutations additionally carry an auxiliary lemma naming the mismatch; there are 55 such lemmas across the corpus. Verse 1.1’s module ends as follows:

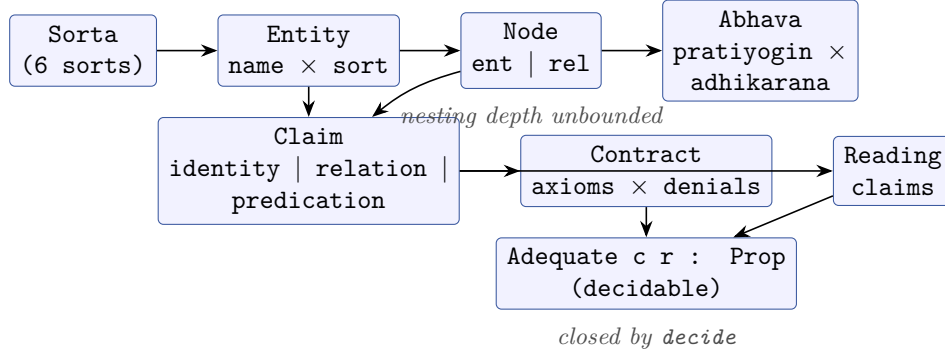


Figure 4: Type structure of the kernel. Relations are first-class relata via **Node**; absence is typed data; adequacy is a decidable proposition over a reading and a contract.

```

theorem accepted_adequate : contract.Adequate accepted := by decide
-- "The imperishable linguistic structure underlying words..."
theorem naive_linguistic_structure_inadequate :
  ¬ contract.Adequate naive_linguistic_structure := by decide
theorem naive_linguistic_structure_sort_error :
  sabdatattva_naive ≠ sabdatattva := by decide
-- "Brahman transforms itself into the world of objects."
theorem parinama_transformation_inadequate :
  ¬ contract.Adequate parinama_transformation := by decide

```

Listing 2: Generated theorems for VP 1.1 (excerpt).

The build gate is binary and global: **lake build** must succeed with zero **sorry** across all 144 modules, 549 theorems at present. A counterexample that stops compiling is a regression, caught in CI.

7 The Agentic Formalization Loop

7.1 Architecture

Contracts were authored by an LLM agent swarm: one agent per verse, each given the verse record, the schema, an exemplar, and the validator as its exit gate. Rounds of 9 to 30 agents ran with an adjudication pass between rounds and an append-only ledger recording every event (96 events: 29 drafts, 40 acceptances, 9 gate runs, 6 validations, 3 rejections, 3 repairs, 5 milestones, 1 publication). The division of labor is the point: agents propose; gates dispose. Figure 5 shows one round.

7.2 What the gates caught

Four episodes illustrate the gate stack functioning under real failure. Table 2 summarizes.

(i) *Lazy counterexamples*. Early agents produced rejected readings that merely restated axioms. The corresponding inadequacy theorems would have been false, and the Lean build failed. The validator was hardened to require provable failure in the registered mode, and three contracts were repaired.

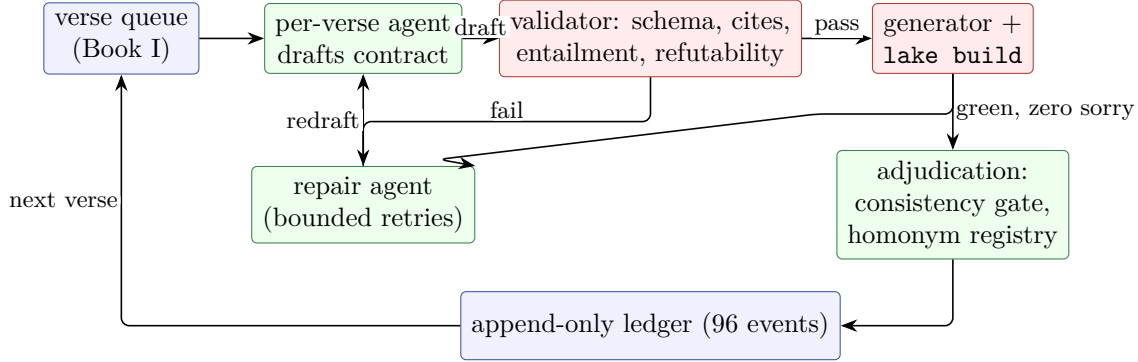


Figure 5: One round of the formalization loop. Every acceptance passes the validator, the Lean kernel, and the cross-contract consistency gate; every event is appended to the ledger.

Table 2: Errors caught mechanically during formalization. A lazy counterexample is a rejected reading whose claims were secretly entailed; a fabricated citation is an axiom cite absent from the commentary.

Failure class	Count	Caught by
Lazy counterexamples	3	Lean kernel, then hardened validator
Fabricated / paraphrased citations	5	verbatim-cite gate
Non-ASCII Lean identifiers	2 contracts	Lean parser, then validator
Cross-contract sort drift	89 conflicts	consistency gate
Diacritic name splits (sphoṭa/sphota)	2 episodes	consistency gate

(ii) *Fabricated citations under degradation.* When an API session limit killed 20 of 30 agents mid-round, 9 left unreviewed draft contracts. Five of the nine failed the verbatim-citation gate with paraphrased or invented cites. This is the anti-fabrication architecture functioning as designed: degraded generation was caught by the gate, quarantined, and re-authored.

(iii) *Identifier hygiene.* Lean’s parser rejected IAST diacritics in entity identifiers that Python’s `isidentifier()` accepts. The validator now enforces ASCII identifiers. The incident argues for keeping the end-to-end kernel gate even when a faster mirror exists.

(iv) *Sort drift.* Independent agents assigned different sorts to the same term. Across five adjudication rounds the consistency gate surfaced 89 conflicts; adjudication resolved each as either a correction or genuine polysemy (§8).

8 Ontological Consistency and the Homonym Registry

The corpus-level gate requires that a named entity carry one sort everywhere, unless the exact partition of (verse, sort) uses is registered as polysemy with a gloss and a one-sentence textual justification per sense. The registry holds 41 terms with 91 senses. This is where the method earns its keep philologically: the gate cannot tell drift from doctrine, so every split is a documented editorial ruling.

Three registered splits are load-bearing. *akṣara*: the imperishable Absolute in 1.9 versus phoneme in 1.18–22. This is the pun that the commentarial tradition treats as the argument of the opening verse in miniature; a same-sort rule would have flattened it. *śabda*: linguisticItem

Table 3: Verification results for the complete Brahmakāṇḍa.

Verse units formalized and verified	144 / 144
Entity declarations	772
Contract axioms / denials	610 / 178
Accepted-reading claims (all proved adequate)	449
Rejected readings, contradicted (denial asserted)	153
Rejected readings, unlicensed (no axiom entails)	197
Auxiliary sort-error lemmas	55
Lean theorems (all by decide , zero sorry)	549
Homonym registry (terms / senses)	41 / 91
Python gates (pytest)	180
Ledger events (append-only)	96

canonically, but the Absolute in the fire-in-wood verse (1.46), where the inner word is śabdatattva itself. *sphoṭa*: linguisticItem in Bhartṛhari’s own doctrine, but verses 1.102–1.106 report a rival phonetician terminology in which sphoṭa names the first-produced acoustic sound. Registering that doxographic span as a distinct sense (manifestation) keeps the report from contaminating the doctrine. A further adjudication round registered polysemy for veda (the recited corpus versus the eternal seed-order of 1.132), śruti, vyavasthā, rūpa, sādhu, viparīta, and kārya.

9 Results

9.1 Verification results

Table 3 summarizes the verified state. CI reproduces it from a clean checkout (corpus gates, **lake build**, zero-**sorry** sweep).

9.2 A map of how Book I can be mistranslated

Beyond the counts, the qualitative yield is a machine-checked map of the ways Book I goes wrong in translation. Each of the 350 refutations is a specific, named failure: pariṇāma for vivarta (1.1); grammar demoted from discipline to convention (1.11–1.14); inference granted an autonomy the text denies it (1.30–1.42); sphoṭa temporalized (1.75–1.77); the corrupt form elevated to equal standing with the correct (1.147–1.155). Together the refutations form a negative image of the doctrine, in theorem form. Because each is a compiled theorem, the map regresses: an edit to any contract that would quietly license a refuted reading breaks the build.

9.3 The edition

The edition is deployed as a static site with per-verse pages carrying Devanagari, IAST, translation, commentary, contested notes, and per-verse proof status, with the repository, contracts, kernel, and ledger public.

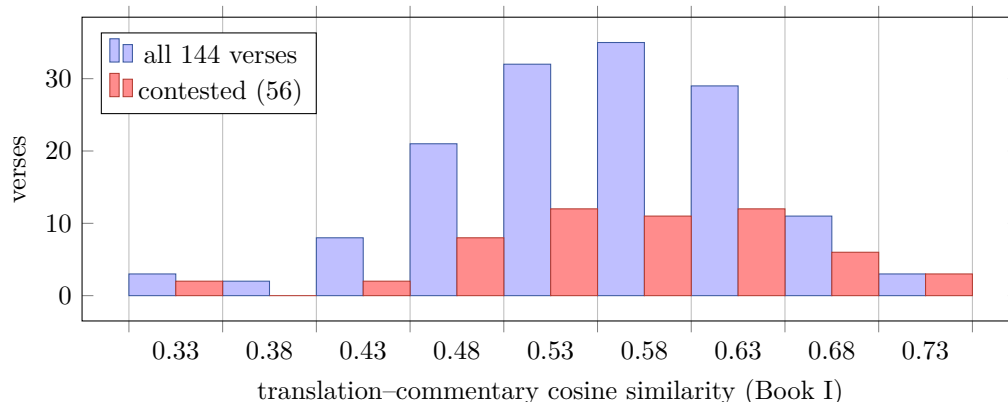


Figure 6: Embedding audit over the Brahmakāṇḍa: distribution of translation-commentary similarity, with the contested subset overlaid. At book scale the contested flag does not separate; the low tail serves as a review queue.

10 Embedding Audit as a Review Queue

Formal verification establishes coherence between translation and contract. A separate instrument screens for drift between translation and commentary prose. Every verse’s translation and commentary are embedded (all-mpnet-base-v2 [Reimers and Gurevych, 2019], mean-pooled, on an NVIDIA GB10) and verses are ranked by cosine similarity.

Within Book I the distribution has mean 0.557 (Figure 6). The audit is reported with a negative finding: at book scale the contested flag does not separate. Contested verses average 0.567 against 0.550 for uncontested verses, and the lowest tail is not enriched (7 of the lowest 20 are contested, against a 38.9% base rate). Across the full 1,796-verse corpus the direction reverses and contested verses shift left (0.578 versus 0.593), but that observation concerns material outside the verified scope. The instrument is accordingly used only as a review queue: low similarity flags verses where translation and commentary may have drifted apart, for editorial attention first. The queue’s head is informative in practice; its lowest-ranked verse (1.74, similarity 0.301) is a contested verse whose commentary is dominated by a doxographic dispute the translation cannot carry.

11 Discussion

What mechanization buys. Three things, on the evidence of this build. *Regression*: interpretive commitments, once contracted, are protected. An edit to verse 1.46’s treatment of the inner word that contradicts 1.1’s identity axioms breaks the build. *Explicitness*: the contract format forces decisions that prose translation lets one defer (is *veda* here the corpus or the eternal seed-order?), and the homonym registry is a by-product catalogue of exactly those decisions. *Adversarial coverage*: requiring every contract to carry provably failing rival readings institutionalizes the question of what it would mean to get each verse wrong. Translators ask that question irregularly; reviewers cannot ask it at scale.

Relation to the companion programmes. The loop’s discipline (propose, gate, ledger, adjudicate) is shared with Active Circuit Discovery and Prabodha [Sathish et al., 2026, Sathish,

2026a]. The instructive difference is the character of the gate. A statistical gate bounds the probability of a wrong acceptance; a type-checking gate makes a wrong acceptance a compile error. The episodes of §7 suggest that when LLM agents author formal content, the strictest available gate should sit at the end of the pipeline even when cheaper mirrors exist, because the mirrors drift (the identifier-hygiene episode) and the agents degrade (the fabricated-citation episode).

Limitations. The entailment relation is deliberately shallow (closure of literal claims under identity symmetry), so adequacy is membership-like rather than inferential. A richer relation, for example transitive sambandha chains or defeasible generics, would strengthen the theorems at the cost of decidability discipline. Contracts formalize the formalizable core of a verse, not its full rhetorical content. The six-sort ontology is coarse; persons, for instance, are shoehorned (the child of 1.121 and 1.151 as epistemic subject). The translation and commentary carry the interpretive standpoint of a single primary source [Subramania Iyer, 1965], and the contracts inherit it. Contracts keyed to rival commentaries (Vṛṣabha, Puṇyarāja) would turn inter-commentarial disagreement into diffable, checkable data, and are the most interesting extension. Finally, the audit instrument is a screen, not a measure of fidelity, and §10 reports where it fails to discriminate.

On agents. The swarm wrote in hours what would take a solo formalizer months, and its failures were institutionally useful: every class of agent error that occurred is now a mechanical gate. The suggested general pattern for LLM-assisted scholarship is to design so that the operative question is never whether the model hallucinated, but whether the artifact passed the gate.

12 Conclusion

This paper introduced commentary-driven translation validation and delivered a machine-verified translation layer for the complete Brahmakāṇḍa of the *Vākyapadīya*: 144 adequacy theorems and 350 compiled refutations over a checksummed edition, produced by a gated agentic loop whose every acceptance is replayable from an append-only ledger. The method treats the prover not as a reader of Sanskrit but as a keeper of commitments. Bhartṛhari holds that the sentence’s meaning is an undivided whole, merely manifested by its parts; a verifier that refuses to let the parts of a translation contradict the whole of its commentary is in that spirit. Books II and III, richer entailment, and multi-commentary contracts are open work.

Availability. Code, corpus, contracts, kernel, proofs, ledger, and the living edition: <https://github.com/SharathSPHD/vakya-vallari>; edition mirrors at <https://vakya-vallari.vercel.app> and <https://huggingface.co/spaces/qbz506/vakya-vallari>.

References

- Ambuda Project. Vidyut: A high-performance Sanskrit toolkit. <https://github.com/ambuda-org/vidyut>, 2024.
- Madeleine Biardeau. *Théorie de la connaissance et philosophie de la parole dans le brahmanisme classique*. Mouton, Paris, 1964.
- George Cardona. *Pāṇini: A Survey of Research*. Mouton, The Hague, 1976.

- Harold G. Coward. *The Sphoṭa Theory of Language: A Philosophical Analysis*. Motilal Banarsidass, Delhi, 1980.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.
- Stephen Diehl. Four-fold logic with Lean. https://www.stephendiehl.com/posts/buddhist_logic_lean/, 2024.
- Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. A distributed platform for Sanskrit processing. In *Proceedings of COLING 2012*, pages 1011–1028, 2012.
- Oliver Hellwig and Sebastian Nehrlich. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. *Proceedings of EMNLP 2018*, pages 2754–2763, 2018.
- Jan E. M. Houben. *The Sambandha-samuddeśa (Chapter on Relation) and Bhartṛhari’s Philosophy of Language*. Egbert Forsten, Groningen, 1995.
- Jan E. M. Houben. Bhartṛhari. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. 2016.
- Daniel H. H. Ingalls. *Materials for the Study of Navya-Nyāya Logic*. Harvard University Press, Cambridge, MA, 1951.
- Paul Kiparsky. On the architecture of Pāṇini’s grammar. *Sanskrit Computational Linguistics, LNCS*, 5402:33–94, 2009.
- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009.
- Bimal Krishna Matilal. *The Navya-Nyāya Doctrine of Negation*. Harvard University Press, Cambridge, MA, 1968.
- Ekansh Panday and Sujata Ghosh. Cubical type theoretic Navya-Nyāya. *arXiv preprint arXiv:2605.12548*, 2026.
- Shashank Pathak. GFLean: An autoformalisation framework for Lean via GF. *arXiv preprint arXiv:2404.01234*, 2024.
- Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *LNCS*, pages 151–166. Springer, 1998.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- SARIT Project. SARIT: Search and retrieval of indic texts. <https://sarit.indology.info/>, 2024. TEI-encoded electronic editions of Sanskrit texts.
- Sharath Sathish. Recognition-gated workspace steering: Pratyabhijñā as an engineering specification for language model control. Preprints.org, 2026a.

- Sharath Sathish. Pramana: Teaching large language models six-phase Navya-Nyāya epistemic reasoning. *arXiv preprint arXiv:2604.04937*, 2026b.
- Sharath Sathish, Mominul Ahsan, and Majid Latifi. Active circuit discovery: A multi-action POMDP agent for causal feature identification in transformer attribution graphs. *Symmetry*, 18(6):1043, 2026. doi: 10.3390/sym18061043.
- Peter M. Scharf. Modeling Pāṇinian grammar. *Sanskrit Computational Linguistics, LNCS*, 5402: 95–126, 2009.
- J. F. Staal. Sanskrit philosophy of language. *Current Trends in Linguistics*, 5:499–531, 1969.
- K. A. Subramania Iyer. *The Vākyapadīya of Bhartr̥hari with the Vṛtti: Chapter I*. Deccan College, Poona, 1965.
- Jean-Baptiste Tristan and Xavier Leroy. Formal verification of translation validators: A case study on instruction scheduling optimizations. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 17–27, 2008.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 32353–32368, 2022.